

GRUPO A: EJERCICIOS INICIALES Y SENCILLOS (Fase de Asimilación)

A.1. Secuenciales (Entrada, Proceso, Salida)

1. Área de un triángulo

□ Algoritmo AreaTriangulo

// Algoritmo para calcular el área de un triángulo a partir de la base y la altura ingresadas por el usuario, empleando la fórmula matemática:

Definir base, altura, area Como Real;

Escribir "Ingrese la base del triángulo (en cm):";

Leer base;

Escribir "Ingrese la altura del triángulo (en cm):";

Leer altura;

area <- (base * altura) / 2;

Escribir "El área del triángulo es: ", area, " cm²";

FinAlgoritmo

□

2. Área de un círculo

Algoritmo AreaCirculo

// Algoritmo para calcular la superficie circular solicitando el radio al usuario. Aplica la constante PI integrada en PSeInt:

Definir radio, area Como Real;

Escribir "Ingrese el radio del círculo:";

Leer radio;

area <- PI * (radio ^ 2);

Escribir "El área del círculo es: ", area;

FinAlgoritmo

3. Ficha de Usuario

Algoritmo FichaUsuario

// Este programa solicita los datos personales del usuario y calcula dinámicamente su edad actual tomando como referencia el año 2026.

Definir nombre, apellidos, direccion, ciudad, cp Como Cadena;

Definir anioNacimiento, edad Como Entero;

Escribir "Introduzca su Nombre:";

Leer nombre;

Escribir "Introduzca sus Apellidos:";

Leer apellidos;

Escribir "Introduzca su Dirección:";

Leer direccion;

Escribir "Introduzca su Ciudad:";

Leer ciudad;

Escribir "Introduzca su Código Postal (CP):";

Leer cp;

Escribir "Introduzca su Año de nacimiento:";

Leer anioNacimiento;

// Suponiendo el año actual de referencia 2026

edad <- 2026 - anioNacimiento;

Escribir "Su nombre es ", nombre, " ", apellidos, " y vive en ", direccion, " de la ciudad ", ciudad, " con el código postal ", cp, ". Tiene ", edad, " años.";

FinAlgoritmo

4. Cálculo de Compra con IVA

Algoritmo CompraConIVA

// Calcula el coste total de una compra aplicando un tipo impositivo estático de IVA del 21%.

Definir precio, cantidad, subtotal, iva, total Como Real;

Escribir "Ingrese el precio del producto:";

Leer precio;

Escribir "Ingrese la cantidad comprada:";

Leer cantidad;

```
subtotal <- precio * cantidad;
iva <- subtotal * 0.21;
total <- subtotal + iva;

Escribir "--- TICKET DE COMPRA ---";
Escribir "Subtotal: ", subtotal, " €";
Escribir "IVA (21%): ", iva, " €";
Escribir "Total a pagar: ", total, " €";
FinAlgoritmo
```

5. Viaje de Estudios

```
Algoritmo ViajeEstudios
  // Calcula la parte proporcional a pagar por cada alumno para el alquiler de un autobús de
  viaje de estudios.
  Definir costeAutobus, pagoPorAlumno Como Real;
  Definir cantidadAlumnos Como Entero;

  Escribir "Ingrese el coste total del autobús:";
  Leer costeAutobus;
  Escribir "Ingrese la cantidad de alumnos que asistirán:";
  Leer cantidadAlumnos;

  Si cantidadAlumnos > 0 Entonces
    pagoPorAlumno <- costeAutobus / cantidadAlumnos;
    Escribir "Cada alumno debe pagar: ", pagoPorAlumno, " €";
  Sino
    Escribir "Error: La cantidad de alumnos debe ser mayor que 0.";
  FinSi
FinAlgoritmo
```

6. Conversor de Millas

```
Algoritmo ConversorMillas
  // Programa de conversión métrica que transforma millas terrestres en kilómetros utilizando
  el factor de equivalencia 1 milla = 1.60934 km.
```

```
Definir millas, kilometros Como Real;

Escribir "Ingrese la distancia en millas terrestres:";
Leer millas;

kilometros <- millas * 1.60934;

Escribir millas, " millas equivalen a ", kilometros, " kilómetros.";
FinAlgoritmo
```

A.2. Condicionales Básicos (Bifurcaciones)

7. Paridad Numérica

```
Algoritmo ParidadNumerica
    // Determina si un número entero es par o impar haciendo uso del operador matemático de
    residuo Mod.
    Definir numeroEntero Como Entero;

    Escribir "Introduce un número entero:";
    Leer numeroEntero;

    Si numeroEntero Mod 2 == 0 Entonces
        Escribir "El número ", numeroEntero, " es PAR.";
    Sino
        Escribir "El número ", numeroEntero, " es IMPAR.";
    FinSi
FinAlgoritmo
```

8. Operaciones Básicas

```
Algoritmo OperacionesBasicas
    // Calcula las operaciones aritméticas de dos números. El programa cuenta con una
    validación lógica para impedir indeterminaciones matemáticas por división entre cero.
    Definir num1, num2, suma, diferencia, producto, division Como Real;

    Escribir "Introduce el primer número:";
```

```
Leer num1;
Escribir "Introduce el segundo número:";
Leer num2;

suma <- num1 + num2;
diferencia <- num1 - num2;
producto <- num1 * num2;

Escribir "Suma: ", suma;
Escribir "Diferencia: ", diferencia;
Escribir "Multiplicación: ", producto;

Si num2 <> 0 Entonces
    division <- num1 / num2;
    Escribir "División: ", division;
Sino
    Escribir "División: No es posible dividir entre cero.";
FinSi
FinAlgoritmo
```

9. Día de la semana

Algoritmo DiaSemana

// Utiliza la estructura de selección múltiple Segun para mapear un número del 1 al 7 con su respectivo día de la semana.

Definir dia Como Entero;

Escribir "Introduce un número del 1 al 7:";

Leer dia;

Segun dia Hacer

- 1: Escribir "Lunes";
- 2: Escribir "Martes";
- 3: Escribir "Miércoles";
- 4: Escribir "Jueves";
- 5: Escribir "Viernes";
- 6: Escribir "Sábado";
- 7: Escribir "Domingo";

De Otro Modo:

Escribir "Número inválido. Debe estar entre 1 y 7.";
FinSegun
FinAlgoritmo

10. Mes del año

Algoritmo MesAnio

 // Mapea un número del 1 al 12 con su correspondiente mes del calendario utilizando la estructura Segun.

 Definir mes Como Entero;

 Escribir "Introduce el número del mes (1 al 12).";

 Leer mes;

 Segun mes Hacer

 1: Escribir "Enero";

 2: Escribir "Febrero";

 3: Escribir "Marzo";

 4: Escribir "Abril";

 5: Escribir "Mayo";

 6: Escribir "Junio";

 7: Escribir "Julio";

 8: Escribir "Agosto";

 9: Escribir "Septiembre";

 10: Escribir "Octubre";

 11: Escribir "Noviembre";

 12: Escribir "Diciembre";

 De Otro Modo:

 Escribir "Error: El número debe estar en el rango [1 - 12].";

 FinSegun

FinAlgoritmo

11. Selección de Área Dinámica

Algoritmo AreaDinamica

 // Permite al usuario elegir qué área calcular, solicitando únicamente las variables necesarias para el tipo de figura seleccionada.

```

Definir opcion Como Caracter;
Definir base, altura, radio, area Como Real;

Escribir "Seleccione la figura para calcular el área ('T' para Triángulo, 'C' para Círculo):";
Leer opcion;

// Convertimos la opción a Mayúscula para evitar fallos de lectura
opcion <- Mayusculas(opcion);

Si opcion == 'T' Entonces
    Escribir "Ingrese la base:";
    Leer base;
    Escribir "Ingrese la altura:";
    Leer altura;
    area <- (base * altura) / 2;
    Escribir "El área del triángulo es: ", area;
Sino
    Si opcion == 'C' Entonces
        Escribir "Ingrese el radio:";
        Leer radio;
        area <- PI * (radio ^ 2);
        Escribir "El área del círculo es: ", area;
    Sino
        Escribir "Opción no reconocida. Debe introducir 'T' o 'C'.";
    FinSi
FinSi
FinAlgoritmo

```

A.3. Repetitivos Básicos (Bucles)

12. Contador Simple

Algoritmo ContadorSimple

// Imprime de forma ascendente los números enteros desde el 1 hasta el límite especificado por el usuario mediante un bucle Para.

Definir limite, i Como Entero;

Escribir "Ingrese el número límite hasta el cual desea contar:";

Leer limite;

```

Si limite >= 1 Entonces
  Para i <- 1 Hasta limite Con Paso 1 Hacer
    Escribir i;
  FinPara
Sino
  Escribir "El limite debe ser mayor o igual a 1.";
FinSi
FinAlgoritmo

```

13. Rango Visual

Versión A: Estructura Mientras

Algoritmo RangoMientras

// Visualiza los números comprendidos entre dos límites ingresados. Implementa dos programas independientes para cubrir las dos estructuras repetitivas solicitadas.

Definir inicio, finNum, aux Como Entero;

Escribir "Ingrese el primer número:";

Leer inicio;

Escribir "Ingrese el segundo número:";

Leer finNum;

// Nos aseguramos de que 'inicio' sea el menor para recorrer de manera ascendente

Si inicio > finNum Entonces

aux <- inicio;

inicio <- finNum;

finNum <- aux;

FinSi

Escribir "Visualizando rango con bucle 'Mientras':";

Mientras inicio <= finNum Hacer

Escribir inicio;

inicio <- inicio + 1;

FinMientras

FinAlgoritmo

Versión B: Estructura Repetir-Hasta

Algoritmo RangoRepetirHasta

// Visualiza los números comprendidos entre dos límites ingresados. Implementa dos programas independientes para cubrir las dos estructuras repetitivas solicitadas.

Definir inicio, finNum, aux Como Entero;

Escribir "Ingrese el primer número:";

Leer inicio;

Escribir "Ingrese el segundo número:";

Leer finNum;

Si inicio > finNum Entonces

 aux <- inicio;

 inicio <- finNum;

 finNum <- aux;

FinSi

Escribir "Visualizando rango con bucle 'Repetir-Hasta:'";

Repetir

 Escribir inicio;

 inicio <- inicio + 1;

Hasta Que inicio > finNum;

FinAlgoritmo

Algoritmo TablaMultiplicarLibre

// Genera la tabla de multiplicar de cualquier número solicitado hasta que el usuario introduzca un valor no válido (cero o negativo).

Definir num, i Como Entero;

Repetir

 Escribir "Introduzca un número para ver su tabla (0 o negativo para salir):";

 Leer num;

Si num > 0 Entonces

 Escribir "--- Tabla del ", num, " ---";

 Para i <- 1 Hasta 10 Con Paso 1 Hacer

 Escribir num, " x ", i, " = ", (num * i);

 FinPara

 Escribir "-----";

```
FinSi
Hasta Que num <= 0;

Escribir "Programa finalizado.";
FinAlgoritmo
```

15. Clasificador de Caracteres

Algoritmo ClasificadorCaracteres

```
// Analiza caracteres uno a uno informando si se trata de vocales o consonantes, hasta que el
usuario digite el carácter '0'.
```

```
Definir car Como Caracter;
```

```
Escribir "Ingrese letras una por una. Escriba '0' para terminar.";
```

```
Leer car;
```

```
car <- Mayusculas(car);
```

```
Mientras car <> '0' Hacer
```

```
Si car == 'A' O car == 'E' O car == 'I' O car == 'O' O car == 'U' Entonces
```

```
    Escribir car, " es VOCAL.";
```

```
Sino
```

```
    // Comprobación de que sea una letra válida (A-Z) excluyendo números/símbolos
```

```
    Si car >= 'A' Y car <= 'Z' Entonces
```

```
        Escribir car, " es CONSONANTE.";
```

```
Sino
```

```
    Escribir car, " no es un carácter alfabético válido.";
```

```
FinSi
```

```
FinSi
```

```
Escribir "Ingrese la siguiente letra (o '0' para salir).";
```

```
Leer car;
```

```
car <- Mayusculas(car);
```

```
FinMientras
```

```
Escribir "Fin del clasificador.";
```

```
FinAlgoritmo
```

16. Acumuladores (Suma y Media)

Versión A: Estructura Mientras

Algoritmo AcumuladorMientras

// Suma y calcula el promedio de un conjunto indeterminado de números ingresados, deteniendo la captura al detectar el número 0.

Definir num, suma, contador Como Entero;

Definir media Como Real;

suma <- 0;

contador <- 0;

Escribir "Ingrese un número (0 para finalizar):";

Leer num;

Mientras num <> 0 Hacer

 suma <- suma + num;

 contador <- contador + 1;

 Escribir "Ingrese otro número (0 para finalizar):";

 Leer num;

FinMientras

Si contador > 0 Entonces

 media <- suma / contador;

 Escribir "Suma acumulada: ", suma;

 Escribir "Media aritmética: ", media;

Sino

 Escribir "No se introdujeron datos numéricos válidos.";

FinSi

FinAlgoritmo

Versión B: Estructura Repetir-Hasta

Algoritmo AcumuladorRepetirHasta

// Suma y calcula el promedio de un conjunto indeterminado de números ingresados, deteniendo la captura al detectar el número 0.

Definir num, suma, contador Como Entero;

Definir media Como Real;

suma <- 0;

contador <- 0;

```

Repetir
  Escribir "Ingrese un número (0 para finalizar):";
  Leer num;
  Si num <> 0 Entonces
    suma <- suma + num;
    contador <- contador + 1;
  FinSi
Hasta Que num == 0;

Si contador > 0 Entonces
  media <- suma / contador;
  Escribir "Suma acumulada: ", suma;
  Escribir "Media aritmética: ", media;
Sino
  Escribir "No se introdujeron datos.";
FinSi
FinAlgoritmo

```

17. Potencia

Algoritmo Potencialterativa

// Calcula el resultado de elevar una base real a un exponente entero utilizando un bucle Para de manera iterativa (multiplicaciones sucesivas).

Definir base, resultado Como Real;

Definir exponente, i Como Entero;

Escribir "Ingrese la base (Real):";

Leer base;

Escribir "Ingrese el exponente (Entero no negativo):";

Leer exponente;

resultado <- 1;

Si exponente >= 0 Entonces

Para i <- 1 Hasta exponente Con Paso 1 Hacer

resultado <- resultado * base;

FinPara

Escribir base, "^", exponente, " = ", resultado;

```
Sino
  Escribir "Error: El exponente debe ser un entero positivo o cero.";
FinSi
FinAlgoritmo
```

18. Potencia hasta negativo

Algoritmo PotenciaHastaNegativo

// Variante del ejercicio de potencia iterativa. Se ejecuta en un bucle continuo hasta que el usuario ingrese un exponente negativo.

```
Definir base, resultado Como Real;
Definir exponente, i Como Entero;
```

Repetir

```
  Escribir "Ingrese la base real:";
  Leer base;
  Escribir "Ingrese el exponente entero (Exponente negativo para salir):";
  Leer exponente;
```

Si exponente ≥ 0 Entonces

```
  resultado <- 1;
  Para i <- 1 Hasta exponente Con Paso 1 Hacer
    resultado <- resultado * base;
  FinPara
  Escribir "Resultado: ", base, "^", exponente, " = ", resultado;
  Escribir "-----";
FinSi
```

Hasta Que exponente < 0 ;

```
Escribir "Bucle detenido. Saliendo del programa.";
FinAlgoritmo
```

A.4. Cadenas, Funciones Básicas y Memoria Lineal Inicial

19. Búsqueda del Menor

Algoritmo BusquedaMenor

// Almacena 10 números proporcionados por el usuario en un arreglo y localiza el valor mínimo del grupo.

Definir numeros, i, menor Como Entero;

Dimension numeros[10];

Escribir "Ingrese 10 números enteros:";

Para i <- 1 Hasta 10 Con Paso 1 Hacer

 Escribir "Elemento [", i, "]:";

 Leer numeros[i];

FinPara

// Inicializamos con el primer elemento

menor <- numeros[1];

Para i <- 2 Hasta 10 Con Paso 1 Hacer

 Si numeros[i] < menor Entonces

 menor <- numeros[i];

 FinSi

FinPara

Escribir "El número menor introducido es: ", menor;

FinAlgoritmo

20. Media de Datos

Algoritmo MediaArreglo

// Utiliza un arreglo de 5 elementos reales para almacenar los datos ingresados y calcula su media aritmética leyendo desde el arreglo.

Definir datos Como Real;

Definir i Como Entero;

Definir suma, media Como Real;

Dimension datos[5];

suma <- 0;

Para i <- 1 Hasta 5 Con Paso 1 Hacer

 Escribir "Ingrese el dato número ", i, ":";

 Leer datos[i];

FinPara

```
// Sumamos los elementos del array
```

```
Para i <- 1 Hasta 5 Con Paso 1 Hacer
```

```
    suma <- suma + datos[i];
```

FinPara

```
media <- suma / 5;
```

```
Escribir "La media de los datos introducidos es: ", media;
```

FinAlgoritmo

21. Nombres al azar

Algoritmo NombresAlAzar

```
// Declara de forma estática una colección con 10 nombres y escoge uno de manera aleatoria a través del generador de enteros Azar o Aleatorio.
```

```
Definir listaNombres Como Cadena;
```

```
Definir indiceAleatorio Como Entero;
```

```
Dimension listaNombres[10];
```

```
// Asignación estática
```

```
listaNombres[1] <- "Carlos";
```

```
listaNombres[2] <- "Ana";
```

```
listaNombres[3] <- "Beatriz";
```

```
listaNombres[4] <- "Daniel";
```

```
listaNombres[5] <- "Eduardo";
```

```
listaNombres[6] <- "Gabriela";
```

```
listaNombres[7] <- "Helena";
```

```
listaNombres[8] <- "Iván";
```

```
listaNombres[9] <- "Julia";
```

```
listaNombres[10] <- "Lucas";
```

```
// Generación de un índice aleatorio entre 1 y 10
```

```
indiceAleatorio <- Aleatorio(1, 10);
```

```
Escribir "El nombre seleccionado al azar es: ", listaNombres[indiceAleatorio];
```

FinAlgoritmo

22. Tamaño del Nombre

Algoritmo TamanoNombre

// Solicita nombre y apellidos por separado, realiza la concatenación insertando los espacios correspondientes y calcula la longitud de la cadena resultante.

Definir nombre, ape1, ape2, nombreCompleto Como Cadena;

Definir caracteresTotales Como Entero;

Escribir "Ingrese su Nombre:";

Leer nombre;

Escribir "Ingrese su Primer Apellido:";

Leer ape1;

Escribir "Ingrese su Segundo Apellido:";

Leer ape2;

nombreCompleto <- nombre + " " + ape1 + " " + ape2;

caracteresTotales <- Longitud(nombreCompleto);

Escribir "Nombre concatenado: ", nombreCompleto, "";

Escribir "Número de caracteres (incluyendo espacios vacíos): ", caracteresTotales;

FinAlgoritmo

23. Longitud de Frase

Funcion retorno <- ObtenerLongitud(frase)

Definir retorno Como Entero;

retorno <- Longitud(frase);

FinFuncion

Algoritmo LongitudFrasePrincipal

// Pide una frase al usuario y delega la responsabilidad de contar su longitud a una función personalizada.

Definir textoUsuario Como Cadena;

Definir tamano Como Entero;

Escribir "Introduzca una frase:";

Leer textoUsuario;

```
tamano <- ObtenerLongitud(textoUsuario);  
  
Escribir "La longitud de la frase procesada por la función es de: ", tamano, " caracteres.";   
FinAlgoritmo
```

24. Contador de Vocales

Algoritmo ContadorVocales

```
// Analiza iterativamente los caracteres de un texto introducido e incrementa un acumulador  
al detectar vocales.
```

```
Definir frase, letra Como Cadena;  
Definir i, totalVocales Como Entero;
```

```
Escribir "Escriba una frase para analizar sus vocales:";  
Leer frase;
```

```
totalVocales <- 0;  
frase <- Mayusculas(frase); // Homogeneizar a mayúsculas
```

```
Para i <- 1 Hasta Longitud(frase) Con Paso 1 Hacer
```

```
    letra <- Subcadena(frase, i, i);
```

```
    Si letra == 'A' O letra == 'E' O letra == 'I' O letra == 'O' O letra == 'U' Entonces
```

```
        totalVocales <- totalVocales + 1;
```

```
    FinSi
```

```
FinPara
```

```
Escribir "La frase introducida contiene ", totalVocales, " vocales.";   
FinAlgoritmo
```

25. Número de Palabras

Algoritmo NumeroPalabras

```
// Determina el número de palabras en un texto contando los espacios en blanco que dividen  
los términos gramaticales.
```

```
Definir frase, letra Como Cadena;  
Definir i, palabras, longitudFrase Como Entero;
```

```

Escribir "Escriba una frase:";
Leer frase;

longitudFrase <- Longitud(frase);
palabras <- 0;

Si longitudFrase > 0 Entonces
    // Partimos de que hay al menos 1 palabra si la frase no está vacía
    palabras <- 1;
    Para i <- 1 Hasta longitudFrase Con Paso 1 Hacer
        letra <- Subcadena(frase, i, i);
        // Si el carácter actual es un espacio y el siguiente no lo es (evitando dobles espacios)
        Si letra == " " Y (i < longitudFrase Y Subcadena(frase, i + 1, i + 1) <> " ") Entonces
            palabras <- palabras + 1;
        FinSi
    FinPara
FinSi

Escribir "Número de palabras estimadas: ", palabras;
FinAlgoritmo

```

26. Buscador de Letras

```

Algoritmo BuscadorLetras
    // Determina con qué frecuencia se repite una letra específica elegida por el usuario dentro
    de un texto.
    Definir frase, caracterBuscar, charActual Como Cadena;
    Definir i, coincidencias Como Entero;

    Escribir "Escriba una frase de texto:";
    Leer frase;
    Escribir "Ingrese el carácter que desea buscar:";
    Leer caracterBuscar;

    // Homogeneizamos a mayúsculas para evitar problemas de distinción de casos
    frase <- Mayusculas(frase);
    caracterBuscar <- Mayusculas(caracterBuscar);
    coincidencias <- 0;

```

```

Para i <- 1 Hasta Longitud(frase) Con Paso 1 Hacer
  charActual <- Subcadena(frase, i, i);
  Si charActual == caracterBuscar Entonces
    coincidencias <- coincidencias + 1;
  FinSi
FinPara

Escribir "El carácter '", caracterBuscar, "' se encontró ", coincidencias, " veces.";
FinAlgoritmo

```

GRUPO B: EJERCICIOS AVANZADOS Y DE APLICACIÓN (Retos y Perfil Profesional)

B.1. Flujos Lógicos y Condicionales Complejos

27. Cruce Seguro (Algoritmo estructurado del Diagrama de Flujo)

Algoritmo CruceSeguroAvenida

// Descripción del comportamiento lógico y secuencial recomendado para modelar mediante diagrama de flujo cómo cruzar una avenida con semáforo peatonal de forma correcta.

Definir colorSemaforo, vehiculosAproximandose Como Cadena;

Definir cruceExitoso Como Logico;

cruceExitoso <- Falso;

Repetir

Escribir "Estás en la acera. Observas el semáforo de peatones.";

Escribir "¿De qué color es la luz del semáforo peatonal? (VERDE / ROJO):";

Leer colorSemaforo;

colorSemaforo <- Mayusculas(colorSemaforo);

Si colorSemaforo == "ROJO" Entonces

Escribir "ALERTA DETENCIÓN: No cruzar. Espera seguro en la acera.";

Escribir "-----";

Sino

Si colorSemaforo == "VERDE" Entonces

Escribir "¿Se aproximan vehículos ignorando la señal o de emergencia? (SI / NO):";

```

Leer vehiculosAproximandose;
vehiculosAproximandose <- Mayusculas(vehiculosAproximandose);

Si vehiculosAproximandose == "NO" Entonces
    Escribir "Cruzas la calzada de forma ágil y con precaución.";
    cruceExitoso <- Verdadero;
Sino
    Escribir "ALERTA DETENCIÓN: Vehículo peligroso aproximándose. Espera.";
    Escribir "-----";
FinSi
Sino
    Escribir "Estado desconocido. Mantente a salvo en la acera.";
FinSi
FinSi
Hasta Que cruceExitoso == Verdadero;

Escribir "¡Has cruzado la avenida con total seguridad!";
FinAlgoritmo

```

28. Conversor de Tiempo Total

```

Algoritmo ConversorTiempoTotal
    // Convierte valores de tiempo en horas, minutos y segundos a su equivalente total medido
    en segundos aplicando la fórmula matemática:
    Definir horas, minutos, segundos, segundosTotales Como Entero;

    Escribir "Introduzca las horas:";
    Leer horas;
    Escribir "Introduzca los minutos:";
    Leer minutos;
    Escribir "Introduzca los segundos:";
    Leer segundos;

    segundosTotales <- (horas * 3600) + (minutos * 60) + segundos;

    Escribir "El tiempo ingresado equivale a: ", segundosTotales, " segundos en total.";
FinAlgoritmo

```

29. Calificación Universitaria Test

Algoritmo CalificacionTest

// Evalúa la puntuación de un examen tipo test de 20 preguntas con criterios de penalización (aciertos: 0.5, fallos: 0.1, sin respuesta: 0).

Definir aciertos, fallos, enBlanco Como Entero;

Definir notaFinal Como Real;

Escribir "Ingrese cantidad de respuestas ACERTADAS:";

Leer aciertos;

Escribir "Ingrese cantidad de respuestas FALLADAS:";

Leer fallos;

Escribir "Ingrese cantidad de respuestas EN BLANCO:";

Leer enBlanco;

// Validamos que el cómputo total de preguntas no exceda las 20 planteadas

Si (aciertos + fallos + enBlanco) == 20 Entonces

 notaFinal <- (aciertos * 0.5) - (fallos * 0.1);

// Controlamos que la nota no resulte negativa

Si notaFinal < 0 Entonces

 notaFinal <- 0;

FinSi

Escribir "--- RESULTADOS DE LA EVALUACIÓN ---";

Escribir "Nota numérica definitiva: ", notaFinal, " / 10.0";

Sino

 Escribir "Error: La suma de aciertos, fallos y blancos debe ser de 20 preguntas.";

FinSi

FinAlgoritmo

30. Calculadora Interactiva

Algoritmo CalculadoraInteractiva

// Una calculadora funcional que procesa una operación aritmética tras solicitar al usuario su selección mediante un menú simplificado.

Definir num1, num2, resultado Como Real;

Definir operacion Como Caracter;

Definir operacionValida Como Logico;

operacionValida <- Verdadero;

Escribir "=== CALCULADORA INTERACTIVA ===";

Escribir "Seleccione una operación básica (+, -, *, /):";

Leer operacion;

Escribir "Introduzca el primer número:";

Leer num1;

Escribir "Introduzca el segundo número:";

Leer num2;

Segun operacion Hacer

"+": resultado <- num1 + num2;

"-": resultado <- num1 - num2;

"*": resultado <- num1 * num2;

"/":

Si num2 <> 0 Entonces

resultado <- num1 / num2;

Sino

Escribir "Error matemático: División entre cero.";

operacionValida <- Falso;

FinSi

De Otro Modo:

Escribir "Símbolo de operación no reconocido.";

operacionValida <- Falso;

FinSegun

Si operacionValida Entonces

Escribir "El resultado es: ", num1, " ", operacion, " ", num2, " = ", resultado;

FinSi

FinAlgoritmo

31. Presupuesto con Desglose (IVA Variable)

Algoritmo PresupuestolvaVariable

// Calcula el coste de adquisición de un artículo aplicando un menú selectivo para determinar el tipo de IVA fiscal aplicable (21%, 10%, o 4%).

Definir producto Como Cadena;
Definir precio, cantidad, subtotal, ivaSeleccionado, implva, total Como Real;
Definir opcionIva Como Entero;
Definir valido Como Logico;

valido <- Verdadero;

Escribir "Nombre del Producto/Servicio:";
Leer producto;
Escribir "Precio Unitario (Base Imponible):";
Leer precio;
Escribir "Cantidad a facturar:";
Leer cantidad;

Escribir "Seleccione la tasa impositiva de IVA aplicable:";
Escribir "1) IVA General (21%);"
Escribir "2) IVA Reducido (10%);"
Escribir "3) IVA Superreducido (4%);"
Leer opcionIva;

Segun opcionIva Hacer

1: ivaSeleccionado <- 0.21;
2: ivaSeleccionado <- 0.10;
3: ivaSeleccionado <- 0.04;

De Otro Modo:

Escribir "Selección impositiva errónea.";
valido <- Falso;

FinSegun

Si valido Entonces

subtotal <- precio * cantidad;
implva <- subtotal * ivaSeleccionado;
total <- subtotal + implva;

Escribir "";
Escribir "=====";
Escribir " TICKET FISCAL ";
Escribir "=====";
Escribir "Concepto: ", producto;
Escribir "Precio/U: ", precio, " € | Cantidad: ", cantidad;
Escribir "Base Imponible: ", subtotal, " €";

```

    Escribir "I.V.A aplicado: ", ivaSeleccionado * 100, "% (" , implva, " €)";
    Escribir "-----";
    Escribir "Importe Total Neto: ", total, " €";
    Escribir "=====";
FinSi
FinAlgoritmo

```

32. Conversor de Moneda Bidireccional con Funciones

```

Funcion usd <- ConvertirEurToUsd(eur)
    Definir usd Como Real;
    usd <- eur * 1.10; // 1 EUR = 1.10 USD
FinFuncion

```

```

Funcion eur <- ConvertirUsdToEur(usd)
    Definir eur Como Real;
    eur <- usd / 1.10;
FinFuncion

```

Algoritmo ConversorMonedaBidireccional

// Realiza cálculos monetarios bidireccionales haciendo uso de subprocesos dedicados para convertir entre divisas (asumiendo un tipo de cambio de 1 EUR = 1.10 USD).

```

Definir producto, moneda Como Cadena;
Definir precio, conversor Como Real;

```

```

Escribir "Nombre del producto:";
Leer producto;
Escribir "Precio del producto:";
Leer precio;
Escribir "¿En qué divisa está expresado el precio? (EUR / USD):";
Leer moneda;

```

```

moneda <- Mayusculas(moneda);

```

```

Escribir "--- INFORME ECONÓMICO ---";
Escribir "Artículo evaluado: ", producto;

```

```

Si moneda == "EUR" Entonces
    conversor <- ConvertirEurToUsd(precio);

```

```

Escribir "Precio Original: ", precio, " EUR";
Escribir "Precio Convertido: ", conversor, " USD";
Sino
  Si moneda == "USD" Entonces
    conversor <- ConvertirUsdToEur(precio);
    Escribir "Precio Original: ", precio, " USD";
    Escribir "Precio Convertido: ", conversor, " EUR";
  Sino
    Escribir "Divisa no compatible en el sistema.";
  FinSi
FinSi
FinAlgoritmo

```

33. Consola de Diagnóstico ASIR

Algoritmo ConsolaASIR

// Simula una pequeña terminal técnica que implementa herramientas de verificación de redes orientadas al perfil profesional de Administración de Sistemas Informáticos en Red.

```

Definir opcion Como Entero;
Definir ipSimulada, cidrVal Como Entero;
Definir velocidadContratada, usoPorcentaje, libres Como Real;

```

```

Repetir
  Escribir "";
  Escribir "===== ";
  Escribir "  CONSOLA DE DIAGNÓSTICO ASIR  ";
  Escribir "===== ";
  Escribir "1) Testear conectividad IP (Ping)";
  Escribir "2) Verificar validez máscara CIDR";
  Escribir "3) Calcular Ancho de Banda Libre";
  Escribir "4) Salir del terminal";
  Escribir "Seleccione una opción:";
  Leer opcion;
  Escribir "-----";

```

Segun opcion Hacer

```

1:
  Escribir "Introduzca el último octeto IP a testear (1-254):";
  Leer ipSimulada;

```

```

Si ipSimulada >= 1 Y ipSimulada <= 254 Entonces
    Escribir "PINGing 192.168.1.", ipSimulada, " con 32 bytes de datos:";
    Escribir "Respuesta desde 192.168.1.", ipSimulada, ": bytes=32 tiempo=12ms TTL=64";
    Escribir "Estado de la interfaz: ACTIVA / CONECTADA";
Sino
    Escribir "Dirección IP fuera de rango local permitido.";
FinSi
2:
Escribir "Ingrese el número de máscara CIDR a evaluar:";
Leer cidrVal;
Si cidrVal >= 8 Y cidrVal <= 30 Entonces
    Escribir "Máscara CIDR /", cidrVal, " es VÁLIDA para direccionamiento IPv4.";
Sino
    Escribir "Máscara CIDR INVÁLIDA (rango idóneo para host entre /8 y /30).";
FinSi
3:
Escribir "Velocidad contratada del canal (en Mbps):";
Leer velocidadContratada;
Escribir "Porcentaje actual de consumo de red (0-100%):";
Leer usoPorcentaje;

Si usoPorcentaje >= 0 Y usoPorcentaje <= 100 Entonces
    libres <- velocidadContratada * (1 - (usoPorcentaje / 100));
    Escribir "Capacidad libre disponible actual: ", libres, " Mbps";
Sino
    Escribir "El porcentaje de uso de canal no es coherente.";
FinSi
4:
Escribir "Cerrando sesión en la terminal...";
De Otro Modo:
    Escribir "Opción del sistema no implementada.";
FinSegun
Hasta Que opcion == 4;
FinAlgoritmo

```

B.2. Estructuras Repetitivas Avanzadas

34. Tablas Múltiples Anidadas

Algoritmo TablasMultiplicarAnidadas

// Utiliza bucles anidados para mostrar las tablas de multiplicar completas del 1 al 5 en una sola secuencia de salida.

Definir tabla, multiplicador Como Entero;

Para tabla <- 1 Hasta 5 Con Paso 1 Hacer

 Escribir "=====";

 Escribir " TABLA DEL ", tabla;

 Escribir "=====";

 Para multiplicador <- 1 Hasta 10 Con Paso 1 Hacer

 Escribir tabla, " x ", multiplicador, " = ", (tabla * multiplicador);

 FinPara

 Escribir ""; // Salto de línea estético

FinPara

FinAlgoritmo

35. Clasificador Estadístico por Intervalos

Algoritmo ClasificadorIntervalos

// Recibe números de manera ininterrumpida hasta que se introduzca un valor negativo, agrupando los datos estadísticamente en intervalos de control.

Definir num, rango1, rango2, fueraRango Como Entero;

rango1 <- 0; // Cuenta números en el intervalo [1, 10]

rango2 <- 0; // Cuenta números en el intervalo [11, 20]

fueraRango <- 0; // Cuenta números fuera de los dos rangos

Repetir

 Escribir "Ingrese un número entero (Valor negativo para detener):";

 Leer num;

 Si num >= 0 Entonces

 Si num >= 1 Y num <= 10 Entonces

 rango1 <- rango1 + 1;

 Sino

 Si num >= 11 Y num <= 20 Entonces

 rango2 <- rango2 + 1;

 Sino

 fueraRango <- fueraRango + 1;

```

        FinSi
    FinSi
FinSi
Hasta Que num < 0;

Escribir "=== METRICAS ESTADISTICAS DE ENTRADA ===";
Escribir "Cantidad de números en el intervalo [1, 10]: ", rango1;
Escribir "Cantidad de números en el intervalo [11, 20]: ", rango2;
Escribir "Cantidad de números fuera de los intervalos: ", fueraRango;
FinAlgoritmo

```

36. Sumatorio y Factorial en Rango

```

Funcion sumat <- CalcularSumatorio(num)
    Definir sumat, i Como Entero;
    sumat <- 0;
    Para i <- 1 Hasta num Con Paso 1 Hacer
        sumat <- sumat + i;
    FinPara
FinFuncion

```

```

Funcion fact <- CalcularFactorial(num)
    Definir fact, i Como Entero;
    fact <- 1;
    Para i <- 1 Hasta num Con Paso 1 Hacer
        fact <- fact * i;
    FinPara
FinFuncion

```

```

Algoritmo SumatorioYFactorialEnRango
    // Calcula la sumatoria y el factorial de todos los números dentro de un intervalo numérico
    utilizando funciones matemáticas dedicadas.
    Definir limInf, limSup, i, resS, resF Como Entero;

    Escribir "Ingrese el límite inferior del rango:";
    Leer limInf;
    Escribir "Ingrese el límite superior del rango:";
    Leer limSup;

```

```

Si limInf > 0 Y limSup >= limInf Entonces
  Para i <- limInf Hasta limSup Con Paso 1 Hacer
    resS <- CalcularSumatorio(i);
    resF <- CalcularFactorial(i);
    Escribir "Número: ", i, " -> Sumatorio: ", resS, " | Factorial: ", resF;
  FinPara
Sino
  Escribir "Asegúrese de ingresar enteros positivos y que el límite superior sea mayor o igual
al inferior.";
FinSi
FinAlgoritmo

```

37. Cuadrados en Rango Numérico

```

Funcion cuad <- ObtenerCuadrado(num)
  Definir cuad Como Entero;
  cuad <- num * num;
FinFuncion

```

Algoritmo CuadradosEnRango
 // Calcula e imprime los cuadrados de los números enteros dentro de un rango utilizando una función modular.

```

  Definir limite, i, resultado Como Entero;

  Escribir "Ingrese el valor límite superior para calcular cuadrados:";
  Leer limite;

  Si limite >= 1 Entonces
    Escribir "Cuadrados calculados en el rango [1 - ", limite, "].";
    Para i <- 1 Hasta limite Con Paso 1 Hacer
      resultado <- ObtenerCuadrado(i);
      Escribir "Número: ", i, " | Cuadrado: ", resultado;
    FinPara
  Sino
    Escribir "El límite de ejecución debe ser un valor igual o superior a 1.";
  FinSi
FinAlgoritmo

```

38. Telemetría ASIR de CPU

Algoritmo TelemetriaCPU

// Recibe lecturas porcentuales de carga de CPU en bucle y las categoriza por niveles de carga, alertando al administrador en caso de detectar valores críticos.

Definir carga, verde, amarillo, rojo Como Entero;

verde <- 0;

amarillo <- 0;

rojo <- 0;

Repetir

Escribir "Ingrese la carga actual del procesador en % (Negativo para suspender monitoreo):";

Leer carga;

Si carga >= 0 Entonces

Si carga <= 100 Entonces

Si carga <= 50 Entonces

verde <- verde + 1;

Escribir "ESTADO: Normal (Verde)";

Sino

Si carga <= 85 Entonces

amarillo <- amarillo + 1;

Escribir "ESTADO: Advertencia (Amarillo)";

Sino

rojo <- rojo + 1;

Escribir "ESTADO CRÍTICO: Sobrecarga (Rojo)";

FinSi

FinSi

Sino

Escribir "Muestra descartada por exceder el 100% de capacidad.";

FinSi

Escribir "-----";

FinSi

Hasta Que carga < 0;

Escribir "=== METRICAS GLOBALES DE SISTEMAS ===";

Escribir "Lecturas en zona segura (<=50%): ", verde;

```
    Escribir "Lecturas en aviso (51-85%): ", amarillo;  
    Escribir "Lecturas en alerta crítica (>85%): ", rojo;  
FinAlgoritmo
```

B.3. Memoria (Arrays), Cadenas Complejas y Modularidad

39. Gestión Completa de Memoria Lineal

Funcion CalcularMayor(array Por Referencia, maxVal Por Referencia, maxPos Por Referencia)

```
    Definir i Como Entero;  
    maxVal <- array[1];  
    maxPos <- 1;  
    Para i <- 2 Hasta 10 Con Paso 1 Hacer  
        Si array[i] > maxVal Entonces  
            maxVal <- array[i];  
            maxPos <- i;  
        FinSi  
    FinPara  
FinFuncion
```

Funcion CalcularMenor(array Por Referencia, minVal Por Referencia, minPos Por Referencia)

```
    Definir i Como Entero;  
    minVal <- array[1];  
    minPos <- 1;  
    Para i <- 2 Hasta 10 Con Paso 1 Hacer  
        Si array[i] < minVal Entonces  
            minVal <- array[i];  
            minPos <- i;  
        FinSi  
    FinPara  
FinFuncion
```

Algoritmo GestionMemoriaLineal

// Valida que un arreglo de 10 elementos contenga únicamente valores en el rango [0 - 150].
A través de parámetros por referencia, determina los valores extremos y sus posiciones de memoria correspondientes.

```
    Definir datos, i, maxV, maxP, minV, minP Como Entero;  
    Definir reiniciar Como Caracter;  
    Dimension datos[10];
```

Repetir

```

Escribir "Ingrese 10 números (Deberán pertenecer estrictamente al rango [0 - 150]):";
Para i <- 1 Hasta 10 Con Paso 1 Hacer
    Repetir
        Escribir "Posición [", i, "]:";
        Leer datos[i];
        Si datos[i] < 0 O datos[i] > 150 Entonces
            Escribir "Valor rechazado. El dato debe estar entre 0 y 150. Pruebe de nuevo.";
        FinSi
    Hasta Que datos[i] >= 0 Y datos[i] <= 150;
FinPara

// Ejecución de rutinas modularizadas por referencia
CalcularMayor(datos, maxV, maxP);
CalcularMenor(datos, minV, minP);

Escribir "--- DIAGNÓSTICO DE MEMORIA ---";
Escribir "Valor Máximo Registrado: ", maxV, " localizado en el índice [", maxP, "];";
Escribir "Valor Mínimo Registrado: ", minV, " localizado en el índice [", minP, "];";
Escribir "-----";

Escribir "¿Desea iniciar de nuevo el proceso de diagnóstico? (S / N):";
Leer reiniciar;
reiniciar <- Mayusculas(reiniciar);
Hasta Que reiniciar == "N";

Escribir "Cierre de gestión de memoria lineal.";
FinAlgoritmo

```

40. Detector de Pares en Entornos Aleatorios

```

Algoritmo DetectorParesAleatorios
    // Genera un arreglo de tamaño dinámico N con valores aleatorios y extrae únicamente los
    números que cumplan con el criterio de paridad.
    Definir nMax, i Como Entero;
    Escribir "Indique el tamaño que desea asignar a la memoria del vector (N):";
    Leer nMax;

    Si nMax > 0 Entonces
        Definir arrayGenerado Como Entero;

```

```

Dimension arrayGenerado[nMax];

// Rellenado aleatorio seguro entre [0 - 99]
Para i <- 1 Hasta nMax Con Paso 1 Hacer
    arrayGenerado[i] <- Aleatorio(0, 99);
FinPara

Escribir "--- Listado completo de pares detectados ---";
Para i <- 1 Hasta nMax Con Paso 1 Hacer
    Si arrayGenerado[i] Mod 2 == 0 Entonces
        Escribir "Índice [", i, "] -> Contiene el valor Par: ", arrayGenerado[i];
    FinSi
FinPara
Sino
    Escribir "Tamaño de memoria incorrecto.";
FinSi
FinAlgoritmo

```

41. Analizador de Jitter de Red

Algoritmo AnalizadorJitter

// Calcula la fluctuación temporal en la latencia de un canal (Jitter) a partir del análisis de las variaciones extremas en un arreglo de 5 lecturas de ping.

Definir pings, i, pingMax, pingMin, jitter Como Entero;

Dimension pings[5];

Escribir "Capture 5 muestras de latencia Ping (medido en ms):";

Para i <- 1 Hasta 5 Con Paso 1 Hacer

Escribir "Ping [muestreo ", i, "]:";

Leer pings[i];

FinPara

// Inicializamos extremos métricos

pingMax <- pings[1];

pingMin <- pings[1];

Para i <- 2 Hasta 5 Con Paso 1 Hacer

Si pings[i] > pingMax Entonces

pingMax <- pings[i];

```

FinSi
Si pings[i] < pingMin Entonces
    pingMin <- pings[i];
FinSi
FinPara

jitter <- pingMax - pingMin;

Escribir "--- INFORME TELEMETRIA DE JITTER ---";
Escribir "Latencia máxima registrada: ", pingMax, " ms";
Escribir "Latencia mínima registrada: ", pingMin, " ms";
Escribir "Jitter de canal resultante: ", jitter, " ms";
FinAlgoritmo

```

42. Detector de Palíndromos

Algoritmo DetectorPalindromos

// Compara los caracteres de una palabra o frase desde los extremos hacia el centro para determinar si se lee de igual forma de izquierda a derecha que de derecha a izquierda.

Definir frase, fraseLimpia, letra Como Cadena;

Definir i, len Como Entero;

Definir esPalindromo Como Logico;

Escribir "Ingrese la palabra o frase a evaluar:";

Leer frase;

frase <- Mayusculas(frase);

fraseLimpia <- "";

// Limpieza previa: omitimos espacios vacíos para evaluar palíndromos complejos

Para i <- 1 Hasta Longitud(frase) Con Paso 1 Hacer

letra <- Subcadena(frase, i, i);

Si letra <> " " Entonces

fraseLimpia <- fraseLimpia + letra;

FinSi

Para

len <- Longitud(fraseLimpia);

esPalindromo <- Verdadero;

```

// Verificación por extremos contrapuestos
Para i <- 1 Hasta Trunc(len / 2) Con Paso 1 Hacer
    Si Subcadena(fraseLimpia, i, i) <> Subcadena(fraseLimpia, len - i + 1, len - i + 1) Entonces
        esPalindromo <- Falso;
    FinSi
FinPara

Si esPalindromo Y len > 0 Entonces
    Escribir "La frase es un PALÍNDROMO.";
Sino
    Escribir "La frase NO es un palíndromo.";
FinSi
FinAlgoritmo

```

43. Extractor de Subcadenas

```

Algoritmo ExtractorSubcadenas
    // Extrae un fragmento específico de un texto delimitado por los índices de inicio y fin
    // ingresados, validando previamente que los rangos se encuentren dentro del tamaño del texto.
    Definir frase, subcadenaResultado Como Cadena;
    Definir posInicio, posFin, tamano Como Entero;

    Escribir "Escriba una frase completa:";
    Leer frase;
    Escribir "Indique el índice de inicio de extracción:";
    Leer posInicio;
    Escribir "Indique el índice de finalización de extracción:";
    Leer posFin;

    tamano <- Longitud(frase);

    // Bloque de validación condicional compleja
    Si posInicio >= 1 Y posFin <= tamano Y posInicio <= posFin Entonces
        subcadenaResultado <- Subcadena(frase, posInicio, posFin);
        Escribir "Fragmento recuperado con éxito: ", subcadenaResultado, "";
    Sino
        Escribir "Error de índice: Compruebe que los límites pertenezcan al rango [1 - ", tamano,
        "].";
    FinSi

```

FinSi
FinAlgoritmo

44. Algoritmo de Cálculo de Letra DNI

Funcion letra <- BuscarLetraDni(numero)

Definir letra Como Caracter;

Definir resto Como Entero;

Definir mapaLetras Como Caracter;

Dimension mapaLetras[23];

// Mapeo oficial de asignación del Ministerio del Interior

mapaLetras[1] <- "T"; mapaLetras[2] <- "R"; mapaLetras[3] <- "W";

mapaLetras[4] <- "A"; mapaLetras[5] <- "G"; mapaLetras[6] <- "M";

mapaLetras[7] <- "Y"; mapaLetras[8] <- "F"; mapaLetras[9] <- "P";

mapaLetras[10] <- "D"; mapaLetras[11] <- "X"; mapaLetras[12] <- "B";

mapaLetras[13] <- "N"; mapaLetras[14] <- "J"; mapaLetras[15] <- "Z";

mapaLetras[16] <- "S"; mapaLetras[17] <- "Q"; mapaLetras[18] <- "V";

mapaLetras[19] <- "H"; mapaLetras[20] <- "L"; mapaLetras[21] <- "C";

mapaLetras[22] <- "K"; mapaLetras[23] <- "E";

// El resto oscila entre 0 y 22

resto <- numero Mod 23;

// Se ajusta al índice del arreglo [1 - 23]

letra <- mapaLetras[resto + 1];

FinFuncion

Algoritmo CalculadorDNI

// Calcula la letra correspondiente a un número de DNI español dividiendo el valor numérico entre 23 y asociando el resto con una tabla de caracteres preconfigurada.

Definir numDni Como Entero;

Definir letraCalculada Como Caracter;

Escribir "Introduzca los 8 dígitos numéricos de su documento de identidad DNI:";

Leer numDni;

Si numDni >= 1000000 Y numDni <= 99999999 Entonces

letraCalculada <- BuscarLetraDni(numDni);

```

    Escribir "La letra correspondiente al número de DNI es: ", letraCalculada;
    Escribir "DNI Formateado: ", numDni, "-", letraCalculada;
Sino
    Escribir "Error: Compruebe el tamaño de los dígitos numéricos de su DNI.";
FinSi
FinAlgoritmo

```

45. Simulador Integral Lotería Primitiva

Algoritmo SimuladorLotería

```

    // Simula el sorteo de la lotería Primitiva utilizando dos estructuras paralelas para registrar y
    comparar la apuesta del usuario con la combinación ganadora generada de forma aleatoria.

```

```

    Definir apuesta, combinacionGanadora, i, j, aciertos, reintegroUsuario, reintegroGanador
    Como Entero;

```

```

    Definir repetido Como Logico;

```

```

    Dimension apuesta[6], combinacionGanadora[6];

```

```

    aciertos <- 0;

```

```

    Escribir "=== REGISTRO DE APUESTA DE LOTERÍA ===";

```

```

    Para i <- 1 Hasta 6 Con Paso 1 Hacer

```

```

        Repetir

```

```

            Escribir "Seleccione su número ", i, " [Rango 1-49]:";

```

```

            Leer apuesta[i];

```

```

        // Validación de duplicados

```

```

        repetido <- Falso;

```

```

        Para j <- 1 Hasta i - 1 Con Paso 1 Hacer

```

```

            Si apuesta[i] == apuesta[j] Entonces

```

```

                repetido <- Verdadero;

```

```

            FinSi

```

```

        FinPara

```

```

    Si apuesta[i] < 1 O apuesta[i] > 49 Entonces

```

```

        Escribir "Error de rango.";

```

```

    Sino

```

```

        Si repetido Entonces

```

```

            Escribir "No puede duplicar números en su apuesta.";

```

```

        FinSi

```

```

    FinSi
    Hasta Que apuesta[i] >= 1 Y apuesta[i] <= 49 Y repetido == Falso;
FinPara

Repetir
    Escribir "Seleccione su reintegro [Rango 0-9]:";
    Leer reintegroUsuario;
    Hasta Que reintegroUsuario >= 0 Y reintegroUsuario <= 9;

// Sorteo de la combinación ganadora sin duplicados
Para i <- 1 Hasta 6 Con Paso 1 Hacer
    Repetir
        combinacionGanadora[i] <- Aleatorio(1, 49);
        repetido <- Falso;
        Para j <- 1 Hasta i - 1 Con Paso 1 Hacer
            Si combinacionGanadora[i] == combinacionGanadora[j] Entonces
                repetido <- Verdadero;
            FinSi
        FinPara
        Hasta Que repetido == Falso;
    FinPara
    reintegroGanador <- Aleatorio(0, 9);

// Comparación de aciertos en la combinación principal
Para i <- 1 Hasta 6 Con Paso 1 Hacer
    Para j <- 1 Hasta 6 Con Paso 1 Hacer
        Si apuesta[i] == combinacionGanadora[j] Entonces
            aciertos <- aciertos + 1;
        FinSi
    FinPara
FinPara

Escribir "";
Escribir "===== ";
Escribir "      RESULTADOS DEL SORTEO      ";
Escribir "===== ";
Escribir "Su Apuesta: ", apuesta[1], " ", apuesta[2], " ", apuesta[3], " ", apuesta[4], " ", apuesta[5],
" ", apuesta[6], " [R:", reintegroUsuario, "];
Escribir "Ganadora: ", combinacionGanadora[1], " ", combinacionGanadora[2], " ",
combinacionGanadora[3], " ", combinacionGanadora[4], " ", combinacionGanadora[5], " ",
combinacionGanadora[6], " [R:", reintegroGanador, "];

```

```

Escribir "-----";
Escribir "Aciertos de la combinación principal: ", aciertos, " / 6";
Si reintegroUsuario == reintegroGanador Entonces
    Escribir "¡Reintegro acertado! Reembolso garantizado.";
Sino
    Escribir "Reintegro no acertado.";
FinSi
Escribir "===== ";
FinAlgoritmo

```

46. Calculadora Exacta de Edad

```

Funcion CalcularEdadExacta(d1, m1, a1, d2, m2, a2)
    Definir anios, meses, dias Como Entero;

    anios <- a2 - a1;

    // Comprobar si aún no se ha cumplido el año en el mes actual
    Si m2 < m1 O (m2 == m1 Y d2 < d1) Entonces
        anios <- anios - 1;
    FinSi

    // Cálculo simplificado aproximado de desfases de meses y días
    Si m2 >= m1 Entonces
        meses <- m2 - m1;
    Sino
        meses <- 12 + (m2 - m1);
    FinSi

    Si d2 >= d1 Entonces
        dias <- d2 - d1;
    Sino
        meses <- meses - 1;
        // Asignación de ajuste aproximado de días (basado en un mes promedio de 30 días)
        dias <- 30 + (d2 - d1);
        Si meses < 0 Entonces
            meses <- 11;
        FinSi
    FinSi
FinSi

```

```
    Escribir "Edad exacta calculada: ", anios, " años, ", meses, " meses y ", dias, " días.";
FinFuncion
```

Algoritmo CalculadoraExactaDeEdad

```
    // Calcula de manera precisa la edad de una persona considerando diferencias de días y
    meses para evitar los desfases temporales propios de los cálculos simplificados.
```

```
    Definir dNac, mNac, aNac Como Entero;
```

```
    Definir dAct, mAct, aAct Como Entero;
```

```
    Escribir "--- INGRESE FECHA DE NACIMIENTO ---";
```

```
    Escribir "Día:"; Leer dNac;
```

```
    Escribir "Mes:"; Leer mNac;
```

```
    Escribir "Año:"; Leer aNac;
```

```
    Escribir "--- INGRESE FECHA ACTUAL DE REFERENCIA ---";
```

```
    Escribir "Día:"; Leer dAct;
```

```
    Escribir "Mes:"; Leer mAct;
```

```
    Escribir "Año:"; Leer aAct;
```

```
    // Validación temporal lógica
```

```
    Si aAct > aNac O (aAct == aNac Y mAct > mNac) O (aAct == aNac Y mAct == mNac Y dAct >=
    dNac) Entonces
```

```
        CalcularEdadExacta(dNac, mNac, aNac, dAct, mAct, aAct);
```

```
    Sino
```

```
        Escribir "La fecha de nacimiento no puede ser posterior a la fecha actual.";
```

```
    FinSi
```

```
FinAlgoritmo
```